

Лабораторная работа **Работа с файловой системой**

Цель работы: Научиться работать с накопителями информации с помощью терминала.

Задания:

1. Просмотрите список смонтированных файловых систем.
2. Выведите сведения о степени использования разделов жёсткого диска.
Выведите сведения о размере домашнего каталога. Размер должен быть указан в килобайтах или мегабайтах. Выведите, также, сведения о размере корневого каталога, при этом не выходите за пределы файловой системы, на которой находится корневой каталог. Использовать команду `du` (Приложение).
3. Добавьте в виртуальную машину ещё один диск, отформатируйте его в файловую систему в `ext4` и примонтируйте к точке монтирования `/mnt/lab` с помощью `mount`. Проверьте правильность командой `lsblk`.
4. Попробуйте открыть диск в программе для просмотра файлов
5. Отмонтируйте диск
6. Откройте `Gparted` и объясните вывод

Дополнительные задания:

1. Выведите первые 10 строк файла `text.txt` (создать файл и заполнить произвольными данными)
2. Выведите последние 10 строк файла `text.txt`
3. Отсортировать файл `text.txt` и вывести из него только уникальные слова
4. Посчитать количество слов и символов в файле `text.txt`
5. Вывести все строчки, содержащие слово «Кот»

Контрольные вопросы:

1. Что такое файловая система?
2. Какие файловые системы вы знаете и где они используются?
3. Что такое точка монтирования?

Приложение:

Презентация по файловым системам: [https://github.com/AKT-Polina/OSiS/blob/main/Lectons/Файловые системы.pdf](https://github.com/AKT-Polina/OSiS/blob/main/Lectons/Файловые%20системы.pdf)

du опции /путь/к/папке

А вот опции утилиты:

- `-a, --all` - выводить размер для всех файлов, а не только для директорий, по умолчанию размер выводится только для папок;
- `-B, --block-size` - указать единицы вывода размера, доступно: K, M, G, T, P, E, Z, Y для 1024 и KB, MB и так далее для 1000;
- `-c, --total` - выводить в конце общий размер всех папок;
- `-d, --max-depth` - максимальная глубина вложенности директорий;
- `-h, --human-readable` - выводить размер в единицах измерения удобных для человека;
- `--inodes` - выводить информацию об использовании `inode`;
- `-L, --dereference` - следовать по всем символическим ссылкам;
- `-l, --count-links` - учитывать размер файла несколько раз для жестких ссылок;
- `-P, --no-dereference` - не следовать по символическим ссылкам, это поведение используется по умолчанию;
- `-S, --separate-dirs` - не включать размер подпапок в размер папки;
- `--si` - выводить размер файлов и папок в системе `si`, используется 1000 вместо 1024;
- `-s, --summarize` - выводить только общий размер;
- `-t, --threshold` - не учитывать файлы и папки с размером меньше указанного;
- `--time` - отображать время последней модификации для файла или папки, вместо времени модификации можно выводить такие метки: `atime`, `access`, `use`, `ctime`;
- `-X, --exclude` - исключить файлы из подсчёта;
- `-x, --one-file-system` - пропускать примонтированные файловые системы;
- `--version` - вывести версию утилиты.

1 Команды по работе с текстом

`head [-n count] [file...]`

Выводит первые count строк файла (по умолчанию 10).

`tail [-f] [-n count] [file...]`

Выводит последние count строк файла (по умолчанию 10). Если указан ключ -f, то ожидает добавления данных в конец файла и выводит их.

`comm [-123] file1 file2`

Считывает файлы file1 и file2, которые должны быть предварительно отсортированы, и выводит три колонки текста. В первой колонке строки имеющиеся только в file1, во второй имеющиеся только в file2, в третьей имеющиеся в обоих файлах. Параметры -1, -2, -3

позволяют подавить вывод соответствующей колонки.

`cut {-c list|-f list [-d delim ]} [file...]`

Вырезает из каждой строки указанные символы и выводит их. Аргумент list - список чисел и диапазонов чисел разделённых запятыми. Для -c числа указывают номера символов

подлежащих выводу, для -f номера полей. Поля разделены символом delim (по умолчанию символ табуляции).

`sort [-c|-m] [-o output] [-urnb] [file...]`

Производит сортировку строк файлов, их объединение или проверяет отсортирован файл или нет. Значения параметров:

- c - только проверить правильность сортировки;
- m - объединить предварительно отсортированные файлы;
- u - удалять повторяющиеся элементы;
- r - сортировка в обратном порядке;
- n - сортировка чисел;
- b - игнорировать лидирующие пробелы;
- o output - производить вывод в файл output.

`tr [КЛЮЧ]... НАБОР1 [НАБОР2]`

Обрабатывает текст посимвольно. Всего доступно 4 ключа для уточнения операции над символами:

- c - Сначала получить дополнение НАБОРА1;
- d - Удалить знаки из НАБОРА2, не превращать;
- s - Замещать последовательность знаков, которые повторяются, из перечисленных в последнем НАБОРЕ, на один такой знак;
- t - Сначала сократить НАБОР1 до размеров НАБОРА2.

НАБОРЫ указываются как символьные строки. В большинстве случаев символы представляют сами себя. Полный набор опций:

- \NNN - Знак в восьмеричной кодировке (с трех цифр NNN);
- \\ - Обратный слэш;
- \b - Забой;
- \f - Перевод страницы;
- \n - Начать с новой строки;
- \r - Возврат каретки;
- \t - Горизонтальная табуляция;
- \v - Вертикальная табуляция;
- ЗНАК1-ЗНАК2 - Все знаки от ЗНАК1 до ЗНАК2 в порядке возрастания;
- [ЗНАК*] - ЗНАК заполняет НАБОР2 до длины НАБОРА1;
- [ЗНАК*ЧИСЛО] - Указанное ЧИСЛО одинаковых ЗНАКОВ; ЧИСЛО восьмиричных, если начинается с 0;
- [:alnum:] - Все буквы и цифры;
- [:alpha:] - Все буквы;
- [:blank:] - Все горизонтальные пробельные символы;
- [:cntrl:] - Все управляющие знаки;
- [:digit:] - Все цифры;
- [:graph:] - Все печатаемые знаки, исключая пробел;
- [:lower:] - Все маленькие буквы;

- [:print:] - Все печатаемые знаки, включая пробел;
- [:punct:] - Все знаки пунктуации;
- [:space:] - Все вертикальные и горизонтальные пробельные знаки;
- [:upper:] - Все большие буквы;
- [:xdigit:] - Все шестнадцатеричные цифры;
- [=ЗНАК=] - Все знаки, эквивалентные ЗНАКУ;

Превращение осуществляется, если не указано -d для обоих НАБОРОВ. -t можно

использовать только во время превращения. Если нужно, НАБОР2 будет расширен до

размеров НАБОРА1 повторением последнего символа. Лишние символы НАБОРА2 будут пропущены. Гарантированно расширяются в порядке возрастания только [:lower:] и [:upper:].

Использованные символы в НАБОРЕ2 во время превращения можно применять для

определения превращения регистра только в парах. -s использует последний указанный

набор. Уплотнение происходит после превращения или удаления.

wc [-c|-m][-lw][file...]

Читает один или более входных файлов и, по умолчанию, выводит число символов новой строки, слов и байт содержащихся в каждом файле на стандартный вывод.

Значения

параметров:

- c - вывести число байт в каждом входном файле;
- l - вывести число символов новой строки в каждом входном файле;
- m - вывести число символов в каждом входном файле;
- w - вывести число слов в каждом входном файле.

iconv -f codepage1 -t codepage2 [file...]

Конвертирует файлы из кодировки codepage1 в кодировку codepage2 и выводит результат на стандартный вывод. Например, iconv -f windows-1251 -t koi8-r file

перекодирует

файл из кодировки CP1251 в кодировку KOI8-R.

Общепрофессиональный (модуль), летний семестр

2 Регулярные выражения

Регулярные выражения предоставляют механизм для выбора определённых строк из множества заданных. Регулярные выражения имеют контекстно независимый синтаксис,

позволяющий им обрабатывать различные наборы символов с различными правилами

сортировки опираясь на текущие установки локали. Большинство приложений поддерживает

базовые регулярные выражения (BRE). Некоторые приложения поддерживают расширенные

регулярные выражения (ERE).

Далее в этом разделе будут использоваться следующие термины:

Полное регулярное выражение - объединённое множество из одного или более BRE (ERE) определяющее шаблон для выбора строк.

Соответствие. Последовательность из нуля или более символов называется соответствующей BRE или ERE, если символы в последовательности соответствуют тем, которые указаны в шаблоне.

3 Базовые регулярные выражения

3.1 BRE соответствующие единственному символу или сопоставимому элементу B

BRE обычный символ, специальный символ с предшествующей обратной чертой или точка соответствуют одному символу. Выражение в квадратных скобках соответствует единственному символу или сопоставимому элементу. Обычный символ в BRE соответствует

сам себе. Это любой символ из используемого кодового набора, за исключением

специальных символов. Специальные символы имеют особые свойства в некотором контексте. Вне этого контекста, либо предварённые обратной чертой, специальные символы соответствуют сами себе. Специальные символы BRE и контексты в которых они имеют специальное значение следующие:

- `\` - специальные, кроме случаев когда используются в выражениях с квадратными скобками;
- `*` - специальный, за исключением использования в квадратных скобках, в качестве первого символа полного BRE (возможно после `^`), первым символом подвыражения (возможно после `^`);
- `^` - специальный когда является первым символом полного BRE или выражения в квадратных скобках;
- `$` - специальный когда является первым символом полного BRE.

Точка, когда используется вне выражения в квадратных скобках, соответствует любому символу из поддерживаемого набора кроме символа с кодом ноль.

3.2 Выражения в квадратных скобках

Выражение в квадратных скобках соответствует единственному сопоставимому элементу содержащемуся в непустом множестве сопоставимых элементов определённом

выражением внутри скобок. К выражениям в квадратных скобках применимы следующие правила и определения:

- Выражение в квадратных скобках может быть либо списком соответствия, либо списком несоответствия. Они состоят из одного или более выражений: подходящие элементы, подходящие символы, классы эквивалентности, классы символов или выражения диапазонов. Специальные символы: `.` `*` `[` `\` теряют своё специальное значение внутри скобок. Последовательности символов `[` `]=` `[:` имеют специальное значение внутри скобок и используются для отделения подходящих символов,

выражений классов эквивалентности, выражений классов символов. За такими последовательностями символов должны следовать правильные выражения, заканчивающиеся, соответственно, символами `]` `=` `:]`.

- Список соответствия определяет список соответствующий любому имеющемуся в нем выражению. Напр., `[abc]` соответствует любому из символов `a`, `b`, `c`.
- Список несоответствия начинается с уголка `^`, и определяет список соответствующий любому символу или подходящему элементу, исключая выражения содержащиеся в списке. Например, `[^abc]` соответствует любому символу или сопоставимому элементу за исключением `a`, `b`, `c`.
- Сопоставимый символ это сопоставимый элемент ограниченный символами `[` и `]`.
- Класс эквивалентности определяет набор сопоставимых элементов. Класс эквивалентности указывается внутри ограничителей `[=` и `=]`.
- Класс символов определяет набор символов. Название класса символов указывается внутри ограничителей `[:` и `:]`. Следующие классы символов определены для всех локалей: `alnum`, `alpha`, `blank`, `cntrl`, `digit`, `graph`, `lower`, `print`, `punct`, `space`, `upper`, `xdigit`.
- Выражение диапазона соответствует любому символу находящемуся между двумя

указанными включительно, с учетом установленного порядка сортировки. При записи начальный и конечный элементы диапазона разделяются знаком минус.

3.3 BRE соответствующие нескольким символам

Следующие правила могут использоваться для построения базовых регулярных выражений соответствующих нескольким символам из BRE соответствующих одному символу:

- Объединение BRE соответствует объединению строк соответствующих каждому из компонентов BRE.
- Могут определяться подвыражения, путём заключения BRE между парами символов `\(` и `\)`. Такие выражения соответствуют тому же, чему и без символов `\(` `\)`.
- Выражение обратной ссылки `n` соответствует той же строке (возможно пустой), что и подвыражение предшествующее `n`. Символ `n` является цифрой от 1 до 9 включительно

и определяет номер подвыражения в строке.

- Если за BRE соответствующим одному символу, подвыражением или обратной ссылкой следует символ звёздочки, то такое выражение (вместе с символом звёздочки) соответствует нулю или более последовательных включений соответствующих данному BRE.

- Если за BRE соответствующим одному символу, подвыражением или обратной ссылкой следует интервальное выражение вида $\{m\}$, $\{m,\}$ или $\{m,n\}$, то такое выражение (вместе с интервальным выражением) соответствует определённому интервальным выражением числу включений соответствующих данному BRE. Выражение $\{m\}$ соответствует точно m включениям, выражение $\{m,\}$ соответствует не менее чем m включениям и выражение $\{m,n\}$ соответствует числу включений от m до n включительно.

3.4 Приоритет BRE

[=] [:] [.]

\<специальный символ>

[]

\(\) \n

* $\{m,n\}$

^ \$

3.5 Привязки

BRE могут быть привязаны к началу или концу строк. Правила привязки следующие:

- Уголок (^), когда является первым символом всего регулярного выражения, соответствует началу строки.

- Знак доллара, когда является последним символом всего регулярного выражения, соответствует концу строки.

- BRE ограниченное символами ^ и \$ соответствует только целой строке.

4 Расширенные регулярные выражения

4.1 Расширенные регулярные выражения соответствующие единственному символу или сопоставимому элементу В ERE обычный символ, специальный символ с предшествующей обратной чертой или точка соответствуют одному символу. Выражение в квадратных скобках соответствует единственному символу или сопоставимому элементу.

Обычный символ в ERE соответствует сам себе. Это любой символ из используемого

кодировочного набора, за исключением специальных символов ERE.

4.2 Специальные символы ERE

Специальные символы ERE имеют особые свойства в некотором контексте. За пределами этого контекста или будучи предварены обратной косой чертой, такие символы соответствуют сами себе. Специальные символы и контексты в которых они имеют

специальное значение следующие:

- . [\ - имеют специальное значение за исключением использования внутри квадратных скобок.

-) - имеет специальное значение когда соответствует открывающей скобке.

- * + ? { | - имеют специальное значение за исключением случаев использования внутри квадратных скобок.

- ^ - имеет специальное значение когда используется в качестве символа привязки, либо является первым символом выражения в квадратных скобках.

- \$ - имеет специальное значение когда используется в качестве символа привязки.

Точка является ERE соответствующим любому символу из поддерживаемого множества за исключением NULL.

4.3 Выражения в квадратных скобках

Правила построения выражений в квадратных скобках не отличаются от BRE.

4.4 ERE соответствующие нескольким символам

Следующие правила используются при построении ERE соответствующих нескольким символам:

- Объединение ERE соответствует объединению последовательностей символов соответствующих каждой компоненте ERE. Объединение ERE заключённое в круглые скобки соответствует тому же, чему и без круглых скобок.

- Когда за ERE соответствующим одному символу или ERE заключённым в фигурные скобки следует знак +, то такое выражение соответствует одному или большему числу последовательных включений соответствующих данному ERE.
- Когда за ERE соответствующим одному символу или ERE заключённым в фигурные скобки следует знак *, то такое выражение соответствует нулю или большему числу последовательных включений соответствующих данному ERE.
- Когда за ERE соответствующим одному символу или ERE заключённым в фигурные скобки следует знак ?, то такое выражение соответствует нулю или одному включению соответствующему данному ERE.
- Когда за ERE соответствующим одному символу или ERE заключённым в фигурные скобки следует интервальное выражение вида {m}, {m,} или {m,n}, то такое выражение соответствует определённому интервальным выражением числу последовательных включений соответствующих данному ERE.

4.5 Альтернативные ERE

Два ERE разделённые символом вертикальной черты (|) соответствуют строке, соответствующей одному из данных ERE.

4.6 Приоритеты ERE

[= =] [: :] [. .]

\<special character>

[]

()

* + ? {m,n}

^ \$

|

4.7 Привязки ERE

ERE могут быть привязаны к началу и концу строки. Для привязки используются символы ^ и \$.

- Знак ^, за пределами выражения в квадратных скобках, привязывает начинающееся с него выражение или подвыражение к началу строки.
- Знак \$, за пределами выражения в квадратных скобках, привязывает заканчивающееся им ERE к концу строки.

5 Утилита grep

Формат командной строки:

grep [-Ecinlv] [-е шаблоны...] [-f файл_шаблонов]... файл...

Производит поиск в указанных файлах или во входном потоке, отбирая строки

в которых имеется соответствие одному или более из указанных шаблонов. Аргумент шаблоны

является списком регулярных выражений разделённых символами новой строки. Аргумент файл_шаблонов определяет файл содержащий список шаблонов. Значения прочих параметров следующие:

- E - использовать расширенные регулярные выражения
- c - вывести только количество строк удовлетворяющих условию
- i - не различать регистр символов
- n - предварить каждую выводимую строку её номером в файле
- l - вывести только имена файлов содержащих строки удовлетворяющие условию
- v - вывести строки не удовлетворяющие ни одному из шаблонов

6 Поточковый редактор sed

Формат командной строки:

sed [-n] сценарий [файл ...]

sed [-n] [-е сценарий]... [-f файл_сценария]... [файл ...]

Sed - потоковый редактор считывающий строки из текстовых файлов или со стандартного ввода, изменяющий их, в зависимости от команд редактирования указанных

в параметре сценарий или в файле файл_сценария и выводящий результат на стандартный вывод. Ключ -n подавляет вывод всех строк файла (производимый по умолчанию). Каждый сценарий (параметр сценарий) состоит из команд редактирования (по одной в строке)

следующего вида:

[адрес!,[адрес3]]команда[аргументы]

допускаются пробельные символы перед первым адресом и перед командой. В нормальном режиме sed циклически копирует входную строку в рабочую область (в случае, если там ничего не осталось от предыдущей команды), применяет к ней последовательно все команды сценария, чьи адресные выражения содержат данную строку, выводит результат на стандартный вывод (если не указан ключ -n) и очищает рабочую область. Некоторые команды используют область сохранения, чтобы запомнить всю рабочую область или ее часть для последующего использования. Рабочая область и область сохранения способны хранить не менее 8192 байт. Адрес либо пустое поле, либо десятичное число, означающее номер входной строки, либо знак доллара (\$), соответствующий последней строке, либо контекстный адрес (который состоит из регулярного выражения предварённого и заканчивающегося некоторым ограничителем, обычно косой чертой). Командная строка не содержащая адреса применяется к каждой строке. Командная строка содержащая один адрес применяется к каждой строке соответствующей адресу. Командная строка содержащая два адреса применяется ко всем строкам начиная с соответствующей первому адресу и заканчивая соответствующей второму. Начиная с первой строки, следующей за указанным диапазоном sed снова проверяет строки на соответствие первому адресу и, в случае нахождения соответствия, процесс повторяется. Команды могут применяться к строкам не входящим в диапазон при помощи команды отрицания !.

Sed поддерживает базовые регулярные выражения со следующими дополнениями:

- В контекстных адресах, конструкция cREc, где c любой символ, отличный от обратной косой черты и символа новой строки, идентична /RE/.
- Последовательность \n соответствует символу новой строки.

В следующем списке команд, максимальное число адресов для команды определено последовательностями [0addr], [1addr] и [2addr]. Аргумент text состоит из одной или более строк. Каждый символ новой строки должен быть предварён обратной косой чертой.

Прочие символы новой строки удаляются.

Общепрофессиональный (модуль), летний семестр

[2addr]command-list - выполнять command-list, для указанных строк.

[1addr]a\text - вывести текст перед обработкой указанной строки.

[2addr]b [метка] - перейти к команде :, содержащей указанную метку. Если метка пуста, перейти в конец сценария.

[2addr]c\text - удалить рабочую область. При пустом или одном адресе или в конце двухадресного диапазона вывести текст.

[2addr]d - удалить рабочую область.

[2addr]D - удалить начало рабочей области, до первого перевода строки включительно.

[2addr]g - заменить содержимое рабочей области содержимым области сохранения.

[2addr]G - добавить к содержимому рабочей области символ новой строки с последующим содержимым области сохранения.

[2addr]h - заменить содержимое области сохранения содержимым рабочей области.

[2addr]H - добавить к содержимому области сохранения содержимое рабочей области.

[1addr]i\text - вывести текст.

[2addr]l - вывести содержимое рабочей области, заменяя символы табуляции, перевода строки, обратной косой черты и т.п. на пары символов \,a ,\b ,\f ,\r ,\t ,\v. Прочие непечатаемые символы заменяются трёхзначными восьмеричными числами предварёнными обратной косой чертой.

[2addr]n - вывести содержимое рабочей области на стандартный вывод и считать в рабочую область следующую входную строку.

[2addr]N - добавить к содержимому рабочей области следующую входную строку. При этом изменяется текущий номер строки.

[2addr]p - вывести рабочую область на стандартный вывод.

[2addr]P - вывести начало рабочей области, до первого символа новой строки на

стандартный вывод.

[1addr]q - перейти в конец сценария и выйти.

[2addr]r rfile - скопировать содержимое файла rfile на стандартный вывод до обработки следующей входной строки.

[2addr]s/reg-exp/replacement/flags - подставить replacement вместо подстрок рабочей области, соответствующих регулярному выражению reg-exp. Вместо косой черты можно

использовать любой другой символ. Символ амперсанда & в replacement будет заменён

строкой, соответствующей регулярному выражению. Символы \n, где n цифра, будут

заменены соответствующей обратной ссылкой. Могут использоваться следующие флаги:

- n - заменять n-ое вхождение регулярного выражения.
- g - заменить все вхождения регулярного выражения.
- p - выводить содержимое рабочей области если произведена замена.
- w wfile - добавить содержимое рабочей области в файл wfile если произведена замена.

[2addr]t [label] - перейти к команде :, содержащей метку label, если с момента последнего чтения входной строки или последнего выполнения команды t в буфере

выполнялись подстановки. Если метка опущена, перейти в конец сценария.

[2addr]w wfile - добавить содержимое рабочей области к файлу wfile.

[2addr]x - обменять содержимое рабочей области и области сохранения.

[2addr]y/string1/string2/ - заменить все включения символов содержащихся в строке string1 соответствующими символами строки string2. Строка string1 не должна содержать повторяющиеся символы, длины строк должны совпадать.

[2addr]!command - применить команду только к тем строкам, которые не принадлежат указанным адресам.

[0addr]: label - команда ничего не делает. Содержит метку, на которую можно перейти командами b и t.

[1addr]= - выводит номер текущей строки.

[0addr] - пустая команда.

[0addr]# - символ # и остальные, до конца строки, игнорируются (комментарий).